Minimally Supervised Categorization of Text with Metadata

Yu Zhang[1*], Yu Meng[1*], Jiaxin Huang[1], Frank F. Xu[2], Xuan Wang[1], Jiawei Han[1]
[1]Department of Computer Science, University of Illinois at Urbana-Champaign, IL, USA
[2]Language Technologies Institute, Carnegie Mellon University, PA, USA
[1]{yuz9, yumeng5, jiaxinh3, xwang174, hanj}@illinois.edu, [2]frankxu@cmu.edu

ABSTRACT

Document categorization, which aims to assign a topic label to each document, plays a fundamental role in a wide variety of applications. Despite the success of existing studies in conventional supervised document classification, they are less concerned with two real problems: (1) *the presence of metadata*: in many domains, text is accompanied by various additional information such as authors and tags. Such metadata serve as compelling topic indicators and should be leveraged into the categorization framework; (2) *label scarcity*: labeled training samples are expensive to obtain in some cases, where categorization needs to be performed using only a small set of annotated data. In recognition of these two challenges, we propose MetaCat, a minimally supervised framework to categorize text with metadata. Specifically, we develop a generative process describing the relationships between words, documents, labels, and metadata. Guided by the generative model, we embed text and metadata into the same semantic space to encode heterogeneous signals. Then, based on the same generative process, we synthesize training samples to address the bottleneck of label scarcity. We conduct a thorough evaluation on a wide range of datasets. Experimental results prove the effectiveness of MetaCat over many competitive baselines.

ACM Reference Format:
Yu Zhang[1*], Yu Meng[1*], Jiaxin Huang[1], Frank F. Xu[2], Xuan Wang[1], Jiawei Han[1]. 2020. Minimally Supervised Categorization of Text with Metadata. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '20), July 25–30, 2020, Virtual Event, China.* ACM, New York, NY, USA, 10 pages. https://doi.org/10.1145/3397271.3401168

1 INTRODUCTION

Our daily life is surrounded by a wealth of text data, ranging from news articles to social media and scientific publications. Document categorization (i.e., assigning a topic label to each document) serves as a critical first step towards organizing, searching and analyzing such a vast spectrum of text data. Many real applications, such as sentiment analysis [35] and location prediction [3], can also be cast as a document categorization task.

Although deep neural models [12, 38] equipped with word embeddings [25] and pre-trained language models [4] have achieved superior performance on document categorization, existing studies are less concerned with two problems in real applications: (1) *The presence of metadata*: metadata prevalently exists in many data sources, especially social media platforms. For example, each tweet is associated with a Twitter user (i.e., its creator) and several hashtags; each Amazon/Yelp review has its product and user information. Such metadata makes each "document" a complex object beyond plain text. As a result, heterogeneous signals should be leveraged in the categorization process. (2) *Label scarcity*: conventional supervised text classification methods [12, 17, 29, 38] rely on a sufficient number of labeled documents as training data. However, annotating enough data for training classification models could be expensive. Moreover, manual labeling requires technical expertise in some domains (e.g., arXiv papers and GitHub repositories), which incurs additional cost. In these cases, it would be favorable to perform categorization using only a small set of training samples that an individual user can afford to provide.

Combining the statements above, we define our task as *minimally supervised categorization of text with metadata*. There exist previous attempts that incorporate *metadata* in text categorization [11, 26, 31]. For example, Kim et al. [11] customize embeddings, transformation matrices and encoder weights in the neural classifier according to metadata information. While it does improve classification accuracy, the model is designed under the fully supervised setting and requires massive training data. Along another line of work, researchers focus on text classification under *weak supervision*[1] [23, 36]. However, they still view documents as plain text sequences and thus are not optimized under the presence of metadata.

In order to effectively leverage multi-modal signals and scarce labeled data *jointly*, in this paper, we propose a unified, embedding-based categorization framework called MetaCat. The design of MetaCat contains two key ideas: (1) to deal with data heterogeneity, we embed words, documents, labels and various metadata (e.g., users, tags and products) into the same latent space to characterize their relationships; (2) to tackle label scarcity, we generate synthesized training samples based on the learned text and metadata embeddings. The generated data, together with the "real" training data, are used to train a neural classifier. We propose a generative model to simultaneously facilitate these two key ideas. Based on the generative process, our model (1) learns the embedding vectors of all elements via maximum likelihood estimation on the textual and metadata statistics; (2) synthesizes training samples with both text and metadata.

*Equal Contribution.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
SIGIR '20, July 25–30, 2020, Virtual Event, China
© 2020 Association for Computing Machinery.
ACM ISBN 978-1-4503-8016-4/20/07…$15.00
https://doi.org/10.1145/3397271.3401168

[1] *Weak supervision* implies that less than a dozen labeled documents are provided for each category.

(a) GitHub Repository (b) Tweet (c) Amazon Review

Figure 1: Three examples of documents with metadata.

We conduct experiments on five real-world datasets from three different domains (GitHub repositories, tweets and Amazon reviews). The results reveal that MetaCat outperforms various text-based and graph-based benchmark approaches. Moreover, we show (1) the superiority of our embedding module towards heterogeneous network embedding methods [5, 6, 27] and (2) the significant contribution of our generation module under weak supervision.

To summarize, this paper makes the following contributions:

- We formulate the problem of minimally supervised categorization of text with metadata. It poses two unique challenges: data heterogeneity and label scarcity.
- We propose a principled generative process to characterize relationships between words, documents, labels and various metadata.
- We develop a novel framework MetaCat with an embedding module and a generation module to tackle the two challenges, respectively. Both modules are derived from the proposed generative process. We also provide theoretical interpretations of our design based on a spherical probabilistic model.
- We conduct extensive experiments to demonstrate the effectiveness of MetaCat and verify the contribution of both embedding and generation modules.

2 PRELIMINARIES

2.1 Problem Definition

Given a collection of documents $\mathcal{D} = \{d_1, ..., d_{|\mathcal{D}|}\}$ and a label space $\mathcal{L} = \{l_1, ..., l_{|\mathcal{L}|}\}$, text classification aims to assign a class label l_i to each document d_j. To characterize each class $l \in \mathcal{L}$, a set of labeled documents $\mathcal{D}_l \subseteq \mathcal{D}$ is provided as training data. Our problem is different from many previous text classification studies [12, 32, 38] from two perspectives. First, each document d_i is accompanied by some metadata m_i (which will be discussed in Section 2.2). Second, traditional supervised methods require sufficient annotated documents (i.e., $|\mathcal{D}_l|$ is large), while we rely on minimal supervision where $|\mathcal{D}_l|$ is small (e.g., less than a dozen). Formally, we define the problem as follows.

Definition 2.1. (Problem Definition) Given documents $\mathcal{D}$, metadata $\mathcal{M}$, label space $\mathcal{L}$ and a small set of training data $\{\mathcal{D}_l : l \in \mathcal{L}\}$, the task is to assign a label $l_i \in \mathcal{L}$ to each document $d_j \in \mathcal{D}$.

2.2 Text and Metadata

The presence of metadata is common in social media corpora. Let us start from some concrete examples. Figure 1 shows three "documents" extracted from different social media platforms. They contain various types of information, which can be grouped as follows.

Text. Textual information is the main body of a document. Some data may have multiple segments of text (e.g., Description+README in GitHub repositories, and Title+Review in Amazon reviews). To simplify our discussion, we concatenate them together for further utilization.

User/Author. The author of a document is a strong topic indicator because one person often has consistent interests. For example, in Figure 1(a), the user "*tensorlayer*" publishes repositories mostly on deep learning; in Figure 1(b), we may infer that the user have many food-related tweets based on her self introduction.

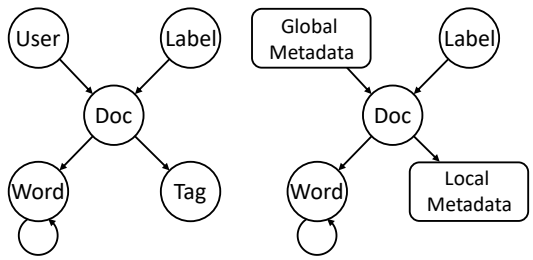
Tag/Hashtag. Tags are a set of concepts describing the documents. Although more concise than text, they still suffer from noise and sparsity. For example, many hashtags in tweets appear only once, and over 70% of GitHub repositories in our collections have no tags. The major difference between words and tags is that words have orders in a sentence (thus local context information needs to be modeled), while tags are swappable.

Product. Product name information is distinctive in Amazon reviews. It is related to both the topic and the sentiment of reviews.

As we can see, the types of metadata are diverse. One can give even more varieties. However, in most cases, topic-indicative metadata can be divided into two major categories according to their relationships with documents.

Global Metadata. Global metadata "causes" the generation of documents. The semantics of a document is based on the semantics of its global metadata. For example, there first exists a user or a product, then some reviews are created by the user/for the product. This generative order cannot be reversed. Therefore, users and products are both global.

Local Metadata. Local metadata "describes" the overall idea of a document. The semantics of local metadata is based on the semantics of its associated document. From this perspective, tags/hashtags are local. We can also say words in text are local, although they are beyond our discussion of "metadata".



(a) GitHub / Tweet (b) The General Case

Figure 2: The generative process of text and metadata. The self loop of "Word" represents the step of words generating contexts.

2.3 The Von Mises-Fisher Distribution

The von Mises-Fisher (vMF) distribution defines a probability density over points on a unit sphere. It is parameterized by a mean direction vector $\boldsymbol{\mu}$ and a concentration parameter κ. Let $\mathbb{S}^{p-1} = \{\boldsymbol{x} \in \mathbb{R}^p : ||\boldsymbol{x}||_2 = 1\}$ denote the p-dimensional unit sphere. Then the probability density function of $\text{vMF}_p(\boldsymbol{\mu}, \kappa)$ is defined as

$$f_{\text{vMF}}(\boldsymbol{x}; \boldsymbol{\mu}, \kappa) = c_p(\kappa) \exp(\kappa \boldsymbol{x}^T \boldsymbol{\mu}), \quad \boldsymbol{x} \in \mathbb{S}^{p-1},$$

where $||\boldsymbol{\mu}||_2 = 1$ and $\kappa > 0$. The normalization constant $c_p(\kappa)$ is given by

$$c_p(\kappa) = \frac{\kappa^{p/2-1}}{(2\pi)^{p/2} I_{p/2-1}(\kappa)},$$

where $I_r(\cdot)$ represents the modified Bessel function of the first kind at order r [7, 19]. Intuitively, the vMF distribution can be interpreted as a normal distribution on the sphere. It characterizes data points concentrating around the mean direction $\boldsymbol{\mu}$, and they are more concentrated if κ is large.

3 METHOD

MetaCat consists of two key modules: embedding learning and training data generation, which are proposed to tackle data heterogeneity and label scarcity, respectively. We start this section by introducing a generative process (Figure 2) that guides the design of these two modules. Following this process, the embedding module learns representation vectors through maximizing the likelihood of observing all data, and the generation module synthesizes training documents with both text and metadata.

For the sake of clarity, we use a specific case (Figure 2(a)) to illustrate our framework. This is also the case of GitHub repositories and tweets. In Section 3.5, we discuss how to generalize the framework (Figure 2(b)).

3.1 The Generative Process

According to Figure 2(a), the generative process can be decomposed into four steps.

User & Label → Document. When a user decides to write a document given a topic, s/he first has an overall idea of what to talk about. This "overall idea" can be represented as document embedding $\boldsymbol{e}_d$, which should be close to user embedding $\boldsymbol{e}_u$ and label embedding $\boldsymbol{e}_l$. Inspired by the softmax function in word2vec [25], we define the generation probability as follows:

$$p(d|u, l) \propto \exp(\boldsymbol{e}_d^T \boldsymbol{e}_u) \cdot \exp(\boldsymbol{e}_d^T \boldsymbol{e}_l). \tag{1}$$

Document → Word. Given the overall idea $\boldsymbol{e}_d$, we can write down words that are coherent with the meaning of the entire document. To encourage such coherence, we have

$$p(w|d) \propto \exp(\boldsymbol{e}_w^T \boldsymbol{e}_d). \tag{2}$$

Document → Tag. Tags can be generated similarly.

$$p(t|d) \propto \exp(\boldsymbol{e}_t^T \boldsymbol{e}_d). \tag{3}$$

Word → Context. Different from tags, words in text carry sequential information. Tang et al. [32] point out that the embedding of a word is related to not only its global document representation (i.e., $\boldsymbol{e}_d$) but also its local context. To be specific, given a sequence of words $w_1 w_2 ... w_n$, the local context of w_i is defined as $C(w_i, h) = \{w_j : i - h \le j \le i + h, i \ne j\}$, where h is the context window size. Mikolov et al. [25] propose the Skip-Gram model in which $C(w_i, h)$ is predicted given the center word w_i. Following [25], we define the generation probability to be

$$p(C(w_i, h)|w_i) \propto \prod_{w_j \in C(w_i, h)} \exp(\boldsymbol{e}'^T_{w_j} \boldsymbol{e}_{w_i}). \tag{4}$$

Note that each word w has two embeddings: $\boldsymbol{e}_w$ when w is viewed as a center word and $\boldsymbol{e}'_w$ when w is a context word [25].

Connections to the vMF Distribution. Now we explain how these conditional probabilities are related to the vMF distribution. Taking $p(w|d)$ as an example, according to Eq. (2), we know that

$$p(w|d) = \frac{\exp(\boldsymbol{e}_w^T \boldsymbol{e}_d)}{\sum_{w' \in \mathcal{V}} \exp(\boldsymbol{e}_{w'}^T \boldsymbol{e}_d)}, \tag{5}$$

where $\mathcal{V}$ is the vocabulary. Following [22], when $|\mathcal{V}|$ goes to infinity and the representation vectors of all elements are assumed to be *unit vectors*, we can generalize Eq. (5) to the continuous case:

$$\lim_{|\mathcal{V}| \to \infty} p(w|d) = \frac{\exp(\boldsymbol{e}_w^T \boldsymbol{e}_d)}{\int_{\mathbb{S}^{p-1}} \exp(\boldsymbol{e}_{w'}^T \boldsymbol{e}_d) d\boldsymbol{e}_{w'}}. \tag{6}$$

To calculate the denominator, one needs to note that the probability density function of vMF distribution integrates to 1 over the whole sphere. Therefore,

$$1 = \int_{\mathbb{S}^{p-1}} f_{\text{vMF}}(\boldsymbol{e}_{w'}; \boldsymbol{e}_d, 1) d\boldsymbol{e}_{w'} = c_p(1) \int_{\mathbb{S}^{p-1}} \exp(\boldsymbol{e}_{w'}^T \boldsymbol{e}_d) d\boldsymbol{e}_{w'}. \tag{7}$$

Combining Eqs. (6) and (7), we get

$$\lim_{|\mathcal{V}| \to \infty} p(w|d) = c_p(1) \exp(\boldsymbol{e}_w^T \boldsymbol{e}_d) = \text{vMF}_p(\boldsymbol{e}_d, 1). \tag{8}$$

Similarly,

$$\begin{aligned} &\lim_{|\mathcal{D}| \to \infty} p(d|u, l) \propto \text{vMF}_p(\boldsymbol{e}_u, 1) \cdot \text{vMF}_p(\boldsymbol{e}_l, 1), \\ &\lim_{|\mathcal{T}| \to \infty} p(t|d) = \text{vMF}_p(\boldsymbol{e}_d, 1), \\ &\lim_{|\mathcal{V}| \to \infty} p(w_j|w_i) = \text{vMF}_p(\boldsymbol{e}_{w_i}, 1), \end{aligned} \tag{9}$$

where $\mathcal{T}$ is the set of tags appearing in the corpus.

The probability $p(d|u, l)$ needs to be elaborated more here. During our embedding step, l is unknown in many cases because only a small proportion of documents have label information. When l

is missing, it is natural to assume that $\boldsymbol{e}_l$ can be any vector on the sphere with equal probability (i.e., $\boldsymbol{e}_l \sim U(\mathbb{S}^{p-1})$). In this case, Eq. (1) becomes

$$\begin{aligned} p(d|u,l) &\propto \mathbb{E}_{\boldsymbol{e}_l \sim U(\mathbb{S}^{p-1})}[\exp(\boldsymbol{e}_d^T \boldsymbol{e}_u) \cdot \exp(\boldsymbol{e}_d^T \boldsymbol{e}_l)] \\ &= \exp(\boldsymbol{e}_d^T \boldsymbol{e}_u) \cdot \mathbb{E}_{\boldsymbol{e}_l \sim U(\mathbb{S}^{p-1})}[\exp(\boldsymbol{e}_d^T \boldsymbol{e}_l)]. \end{aligned} \tag{10}$$

For any fixed $\boldsymbol{e}_d$, using Eq. (7), we have

$$\mathbb{E}_{\boldsymbol{e}_l \sim U(\mathbb{S}^{p-1})}[\exp(\boldsymbol{e}_d^T \boldsymbol{e}_l)] \propto \int_{\mathbb{S}^{p-1}} \exp(\boldsymbol{e}_d^T \boldsymbol{e}_l) d\boldsymbol{e}_l = 1/c_p(1). \tag{11}$$

In other words, this term is the same for any $\boldsymbol{e}_d \in \mathbb{S}^{p-1}$. Therefore, when there is no label information,

$$p(d|u,l) \propto \exp(\boldsymbol{e}_d^T \boldsymbol{e}_u). \tag{12}$$

Our assumption that all embeddings are unit vectors has empirical bases because normalizing embeddings onto a sphere is common practice in natural language processing [13, 37].

3.2 Generation-Guided Embedding Learning

Given the generative process, we are able to learn $\boldsymbol{e}_u$, $\boldsymbol{e}_l$, $\boldsymbol{e}_d$, $\boldsymbol{e}_t$, $\boldsymbol{e}_w$ and $\boldsymbol{e}'_w$ through maximum likelihood estimation.

Likelihood. Assume all embedding vectors are parameters of our generative model. The likelihood of observing the whole corpus (including metadata) is

$$\begin{aligned} \mathcal{J} = &\prod_{d \in \mathcal{D}} p(d|u_d, l_d) \cdot \prod_{d \in \mathcal{D}} \prod_{t \in \mathcal{T}_d} p(t|d) \cdot \\ &\prod_{d \in \mathcal{D}} \prod_{w_i} p(w_i|d) p(C(w_i,h)|w_i), \end{aligned} \tag{13}$$

where u_d is the user creating document d; l_d is the label of document d; $\mathcal{T}_d$ is the set of tags in d. If d is not labeled (i.e., l_d is unknown), according to Eq. (12),

$$p(d|u_d, l_d) = \frac{\exp(\boldsymbol{e}_d^T \boldsymbol{e}_{u_d})}{\sum_{d' \in \mathcal{D}} \exp(\boldsymbol{e}_{d'}^T \boldsymbol{e}_{u_d})}. \tag{14}$$

If d is labeled, according to Eq. (1),

$$\begin{aligned} p(d|u_d, l_d) &= \frac{\exp(\boldsymbol{e}_d^T \boldsymbol{e}_{u_d}) \exp(\boldsymbol{e}_d^T \boldsymbol{e}_{l_d})}{\sum_{d' \in \mathcal{D}} \exp(\boldsymbol{e}_{d'}^T \boldsymbol{e}_{u_d}) \exp(\boldsymbol{e}_{d'}^T \boldsymbol{e}_{l_d})} \\ &\propto \frac{\exp(\boldsymbol{e}_d^T \boldsymbol{e}_{u_d})}{\sum_{d' \in \mathcal{D}} \exp(\boldsymbol{e}_{d'}^T \boldsymbol{e}_{u_d})} \cdot \frac{\exp(\boldsymbol{e}_d^T \boldsymbol{e}_{l_d})}{\sum_{d' \in \mathcal{D}} \exp(\boldsymbol{e}_{d'}^T \boldsymbol{e}_{l_d})}. \end{aligned} \tag{15}$$

Similarly, other conditional probabilities can be derived using Eqs. (2)-(4). Therefore,

$$\begin{aligned} \log \mathcal{J} = &\sum_{u \in \mathcal{U}} \sum_{d \in \mathcal{D}_u} \log \frac{\exp(\boldsymbol{e}_d^T \boldsymbol{e}_u)}{\sum_{d'} \exp(\boldsymbol{e}_{d'}^T \boldsymbol{e}_u)} + \sum_{l \in \mathcal{L}} \sum_{d \in \mathcal{D}_l} \log \frac{\exp(\boldsymbol{e}_d^T \boldsymbol{e}_l)}{\sum_{d'} \exp(\boldsymbol{e}_{d'}^T \boldsymbol{e}_l)} \\ &+ \sum_{d \in \mathcal{D}} \sum_{t \in \mathcal{T}_d} \log \frac{\exp(\boldsymbol{e}_t^T \boldsymbol{e}_d)}{\sum_{t'} \exp(\boldsymbol{e}_{t'}^T \boldsymbol{e}_d)} + \sum_{d \in \mathcal{D}} \sum_{w_i} \log \frac{\exp(\boldsymbol{e}_{w_i}^T \boldsymbol{e}_d)}{\sum_{w'} \exp(\boldsymbol{e}_{w'}^T \boldsymbol{e}_d)} \\ &+ \sum_{d \in \mathcal{D}} \sum_{w_i} \sum_{w_j \in C(w_i,h)} \log \frac{\exp(\boldsymbol{e}'^T_{w_j} \boldsymbol{e}_{w_i})}{\sum_{w'} \exp(\boldsymbol{e}'^T_{w'} \boldsymbol{e}_{w_i})} + \text{const}. \end{aligned} \tag{16}$$

Here $\mathcal{U}$ is the set of users in the dataset; $\mathcal{D}_u$ is the set of documents belonging to user u; $\mathcal{D}_l$ is the set of documents (in the training set) with label l. $\boldsymbol{e}_u$, $\boldsymbol{e}_l$, $\boldsymbol{e}_d$, $\boldsymbol{e}_t$, $\boldsymbol{e}_w$ and $\boldsymbol{e}'_w$ can be learned by maximizing $\log \mathcal{J}$. However, the denominators in Eq. (16) require summing over all documents/tags/words, which is computationally expensive. Following common practice, in our actual computation, we estimate these terms through negative sampling [25, 33].

Comparisons with Heterogeneous Network Embedding. There are various ways to encode multi-modal signals, among which Heterogeneous Information Network (HIN) embedding [5, 6, 27] is commonly used. In fact, if we remove the edge directions in Figure 2(a), it can also be viewed as an HIN schema [30]. We would like to emphasize two key differences between our method and HIN embedding. First, HIN models *connections* between different types of nodes, while our approach models *generative relationships*. Not all connections can be explained as a generative story. From this perspective, our method is more specifically designed to encode text with metadata information. Second, many HIN embedding methods [5, 27] require users to specify a set of meta-paths [30], which is not needed in our approach.

3.3 Training Data Generation

To deal with label scarcity, we consider to generate synthesized training data. In Figure 2, we have proposed a generative process to characterize heterogeneous signals in both text and metadata. Our embedding module follows this process, and so will our generation module. To be specific, given a label l, we first generate a document vector e_d from $p(d|u,l)$ and then sample words and tags one by one from $p(w|d)$ and $p(t|d)$. Formally,

$$p(d, w_{1:n}, t_{1:m}|u,l) = p(d|u,l) \cdot \prod_{i=1}^{n} p(w_i|d) \cdot \prod_{j=1}^{m} p(t_j|d). \tag{17}$$

Unlike in the case of embedding, when we tend to generate new documents, u is not observable. Also, there is no need to generate training data for a specific existing user because our task is to predict the label, not the user, of each document. If u is assumed to be unknown, following the derivation of Eqs. (10)-(12), we have

$$p(d|u,l) \propto \exp(\boldsymbol{e}_d^T \boldsymbol{e}_l). \tag{18}$$

In principle, we are able to generate as many documents as we want. In other words, there is an infinite number of documents (i.e., $\boldsymbol{e}_d$) distributed on the sphere, and we are essentially picking some of them as synthesized training samples. Similar to Eq. (8), by assuming $|\mathcal{D}| \to \infty$, we have

$$p(d|u,l) = \text{vMF}_p(\boldsymbol{e}_l, \kappa). \tag{19}$$

The generation of words and tags, however, is different. The semantics of words and tags are discretely distributed in the latent space. To be specific, not every unit vector can be mapped back to an existing word or tag in the vocabulary. Therefore, we still assume $|\mathcal{V}|$ and $|\mathcal{T}|$ are finite in our generation step, and the generated words or tags must have appeared in $\mathcal{V}$ or $\mathcal{T}$. Then the generation probabilities simply follow Eqs. (2) and (3). In practice, the computation of $\sum_{w' \in \mathcal{V}} \exp(\boldsymbol{e}_{w'}^T \boldsymbol{e}_d)$ could be quite expensive. Therefore, we restrict the word/tag candidates to be the top-τ ones similar with $\boldsymbol{e}_d$ on the sphere (denoted as $\mathcal{N}(\boldsymbol{e}_d)$), and the conditional probabilities will be modified as

$$p(w_i|d) = \frac{\exp(\boldsymbol{e}_{w_i}^T \boldsymbol{e}_d)}{\sum_{w' \in \mathcal{N}(\boldsymbol{e}_d)} \exp(\boldsymbol{e}_{w'}^T \boldsymbol{e}_d)}, \ w_i \in \mathcal{N}(\boldsymbol{e}_d), \tag{20}$$

and

$$p(t_j|d) = \frac{\exp(\boldsymbol{e}_{t_j}^T \boldsymbol{e}_d)}{\sum_{t' \in \mathcal{N}(\boldsymbol{e}_d)} \exp(\boldsymbol{e}_{t'}^T \boldsymbol{e}_d)}, \quad t_j \in \mathcal{N}(\boldsymbol{e}_d). \tag{21}$$

Plugging Eqs. (19), (20) and (21) into Eq. (17) fully specifies our generation step. Each generated document has a label, a sequence of words and a set of tags. We denote the set of synthesized training document for class l as $\mathcal{D}_l^*$.

3.4 Neural Model Training

We now need to feed the synthesized training data $\{\mathcal{D}_l^* : l \in \mathcal{L}\}$, together with the "real" training data $\{\mathcal{D}_l : l \in \mathcal{L}\}$, into a classifier. Here each document (either synthesized or real) can be viewed as a sequence of words $w_{1:n}$ and tags $t_{1:m}$. The input of the classifier is pre-trained embeddings $\{\boldsymbol{e}_w : w \in \mathcal{V}\}$ and $\{\boldsymbol{e}_t : t \in \mathcal{T}\}$. This setting is generic enough so that many neural classifiers, such as CNN [12], HAN [38], CNN-LSTM [43], BiGRU-CNN [34] and DPCNN [10], can be applied in our framework. Developing an advanced neural classifier is not a goal of this paper. Instead, we aim to show the power of our embedding and generation modules. Their contribution to the performance should not be covered by advanced neural architectures when comparing with baselines. Therefore, we choose CNN [12], a simple but widely used model as our text classifier. We also tried HAN [38], a representative RNN-based classifier, but it performs slightly worse than CNN on our datasets.

In the CNN architecture, each document is represented by the concatenation of its word and tag embeddings $\boldsymbol{e}_{1:n+m} = [\boldsymbol{e}_{w_1}, ..., \boldsymbol{e}_{w_n}, \boldsymbol{e}_{t_1}, ..., \boldsymbol{e}_{t_m}]$. There is one convolutional layer, in which a convolution filter is applied to a text region $\boldsymbol{e}_{i:i+h-1}$.

$$c_i = \sigma(\boldsymbol{w}^T \boldsymbol{e}_{i:i+h-1} + \boldsymbol{b}), \tag{22}$$

where σ is the sigmoid function. All c_i's together form a feature map $\boldsymbol{c} = [c_1, ..., c_{n+m-h+1}]$ associated with the filter. Then a max-over-time pooling operation is performed on $\boldsymbol{c}$. In this paper, we use filters with $h = 2, 3, 4$ and 5. For each width h, we generate 20 feature maps. After pooling, the features are passed through a fully connected softmax layer whose output is the probability distribution over labels.

Following [38], we use negative log-likelihood of the correct labels as training loss. Formally, given all training samples (including both real and synthesized ones), assume the i-th one has label $lb(i)$. Then

$$loss = -\sum_i \log q_{i,lb(i)}, \tag{23}$$

where $\boldsymbol{q}_i$ is the neural network output distribution of the i-th instance.

3.5 The General Version of Our Framework

Although we illustrate our framework using Figure 2(a), MetaCat can be easily generalized to any generative process following Figure 2(b). To be specific, let $\mathcal{M}_\mathcal{G}$ be the set of global metadata variables and $\mathcal{M}_\mathcal{L}$ be the set of local metadata variables. (For example, in the cases of GitHub repositories and tweets, $\mathcal{M}_\mathcal{G} = \{user\}$ and $\mathcal{M}_\mathcal{L} = \{tag\}$; in the case of Amazon reviews, $\mathcal{M}_\mathcal{G} = \{user, product\}$ and $\mathcal{M}_\mathcal{L} = \emptyset$.) The generative process can be described as follows:

Global Metadata & Label → Document.

$$p(d|\mathcal{M}_\mathcal{G}, l) \propto \exp(\boldsymbol{e}_d^T \boldsymbol{e}_l) \cdot \prod_{z \in \mathcal{M}_\mathcal{G}} \exp(\boldsymbol{e}_d^T \boldsymbol{e}_z). \tag{24}$$

Document → Word & Local Metadata.

$$p(w, \mathcal{M}_\mathcal{L}|d) \propto \exp(\boldsymbol{e}_w^T \boldsymbol{e}_d) \cdot \prod_{z \in \mathcal{M}_\mathcal{L}} \exp(\boldsymbol{e}_z^T \boldsymbol{e}_d). \tag{25}$$

Word → Context.

$$p(C(w_i, h)|w_i) \propto \prod_{w_j \in C(w_i, h)} \exp(\boldsymbol{e}'^T_{w_j} \boldsymbol{e}_{w_i}). \tag{26}$$

Given the generative process, the derivations of embedding learning and training data generation are quite similar to those in Sections 3.2 and 3.3. We directly show the results here.

Embedding Learning. The log-likelihood is

$$\begin{aligned}\log \mathcal{J} = &\sum_{z \in \mathcal{Z}_\mathcal{G}} \sum_{d \in \mathcal{D}_z} \log \frac{\exp(\boldsymbol{e}_d^T \boldsymbol{e}_z)}{\sum_{d'} \exp(\boldsymbol{e}_{d'}^T \boldsymbol{e}_z)} + \sum_{l \in \mathcal{L}} \sum_{d \in \mathcal{D}_l} \log \frac{\exp(\boldsymbol{e}_d^T \boldsymbol{e}_l)}{\sum_{d'} \exp(\boldsymbol{e}_{d'}^T \boldsymbol{e}_l)} \\ &+ \sum_{d \in \mathcal{D}} \sum_{z \in \mathcal{Z}_d} \log \frac{\exp(\boldsymbol{e}_z^T \boldsymbol{e}_d)}{\sum_{z'} \exp(\boldsymbol{e}_{z'}^T \boldsymbol{e}_d)} + \sum_{d \in \mathcal{D}} \sum_{w_i} \log \frac{\exp(\boldsymbol{e}_{w_i}^T \boldsymbol{e}_d)}{\sum_{w'} \exp(\boldsymbol{e}_{w'}^T \boldsymbol{e}_d)} \\ &+ \sum_{d \in \mathcal{D}} \sum_{w_i} \sum_{w_j \in C(w_i, h)} \log \frac{\exp(\boldsymbol{e}'^T_{w_j} \boldsymbol{e}_{w_i})}{\sum_{w'} \exp(\boldsymbol{e}'^T_{w'} \boldsymbol{e}_{w_i})} + \text{const}.\end{aligned} \tag{27}$$

Here $\mathcal{Z}_\mathcal{G}$ is the set of global metadata instances (e.g., $\mathcal{Z}_\mathcal{G} = \mathcal{U}$ for GitHub repositories and tweets, and $\mathcal{Z}_\mathcal{G} = \mathcal{U} \cup \mathcal{P}$ for Amazon reviews, where $\mathcal{P}$ is the set of products); $\mathcal{D}_z$ is the set of documents belonging to a global metadata instance z; $\mathcal{Z}_d$ is the set of local metadata instances associated with document d (e.g., $\mathcal{Z}_d = \mathcal{T}_d$ for GitHub repositories and tweets, and $\mathcal{Z}_d = \emptyset$ for Amazon reviews). All embeddings can be learned by maximizing this likelihood.

Training Data Generation. Given label l, we first generate e_d from $p(d|\mathcal{M}_\mathcal{G}, l)$ and then sample words w_i and local metadata instances z_j from $p(w|d)$ and $p(z|d)$, respectively.

$$p(d, w_{1:n}, z_{1:m}|\mathcal{M}_\mathcal{G}, l) = p(d|\mathcal{M}_\mathcal{G}, l) \cdot \prod_{i=1}^{n} p(w_i|d) \cdot \prod_{j=1}^{m} p(z_j|d). \tag{28}$$

The generation of documents follows Eq. (19). By assuming $\mathcal{M}_\mathcal{G}$ is not observable and $|\mathcal{D}| \to \infty$, we have

$$p(d|\mathcal{M}_\mathcal{G}, l) = \text{vMF}_p(\boldsymbol{e}_l, \kappa). \tag{29}$$

The generation of words and local metadata follows Eqs. (20) and (21), where we only need to replace t_j with z_j.

4 EXPERIMENTS

In order to provide evidence for the efficacy of MetaCat, we experiment with five datasets collected from different domains. Specifically, we aim to show that (1) MetaCat outperforms benchmark approaches by a clear margin; (2) the proposed generation-guided embedding module captures multi-modal semantics better than existing meta-path based HIN embedding techniques; (3) incorporating synthesized training samples can significantly boost the classification performance.

Table 1: Dataset Statistics.

Dataset	#Document	#Class	#Training	#Testing
GitHub-Bio [42]	876	10	100	776
GitHub-AI [42]	1,596	14	140	1,456
GitHub-Sec	84,950	3	30	84,920
Amazon [20]	100,000	10	100	99,900
Twitter [40]	135,619	9	90	135,529

4.1 Datasets

The five datasets we use are collected from three different sources: GitHub repositories, Amazon reviews and Twitter posts.[2]

- **GitHub-Bio [42].** This dataset is extracted from four bioinformatics venues from 2014 to 2018. Each GitHub repository is associated with a research article published on these venues. The issue section (e.g., sequence analysis, genome analysis, systems biology, etc.) of the article is viewed as the topic label of the associated repository.
- **GitHub-AI [42].** This dataset is collected by the Paper With Code project. It contains a list of GitHub repositories implementing algorithms of various machine learning tasks (e.g., image generation, machine translation, speech recognition, etc.). Each task is viewed as a topic label.
- **GitHub-Sec.** This dataset is obtained from the DARPA SocialSim Challenge. It contains GitHub repositories related to cryptocurrency, cybersecurity or software vulnerability.
- **Amazon [20].** This dataset is a large crawl of Amazon product reviews. The topic label of each review is its product category (e.g., books, home & kitchen, sports & outdoors, etc.). We select 10 large categories and sample 10,000 reviews from each category.
- **Twitter [40].** This dataset contains geo-tagged tweets in New York City during 2014.08.01 – 2014.11.30. The authors link the tweets with Foursquare's POI database, and the topic label of each tweet is the type of the linked POI (e.g., shop & service, college & university, nightlife spot, etc.).

We use 10 documents in each class for training and all the others for testing (i.e., $|\mathcal{D}_l| = 10$). Brief statistics of the five datasets are summarized in Table 1.

4.2 Baseline Methods

We evaluate the performance of MetaCat against both text-based and graph-based benchmark approaches:

- **CNN [12]** is a supervised text classification method. It trains a convolutional neural network with a max-over-time pooling layer.
- **HAN [38]** is a supervised text classification method. It trains a hierarchical attention network and uses GRU to encode word sequences.
- **PTE [32]** is a semi-supervised approach. It constructs a network with three subgraphs (word-word, word-document and word-label) and embeds nodes based on first and second order proximities.
- **WeSTClass [23]** is a weakly supervised text classification approach. It models topic semantics in the word2vec embedding space and applies a pre-training and self-training scheme.
- **PCEM [36]** is a weakly supervised hierarchical text classification method using path cost-sensitive learning. In our problem setting, the label hierarchy has only one layer.
- **BERT [4]** is a state-of-the-art pre-trained language model that provides contextualized word representations. Here we fine-tune BERT under the supervised text classification setting using labeled documents.
- **ESim [27]** is an HIN embedding approach. It learns node embeddings using meta-path guided sequence sampling and noise-contrastive estimation.
- **Metapath2vec [5]** is an HIN embedding approach. It samples node sequences through heterogeneous random walks and incorporates negative sampling.
- **HIN2vec [6]** is an HIN embedding approach that exploits different types of links among nodes.
- **TextGCN [39]** is a semi-supervised text classification approach. It applies graph neural networks on the document-word co-occurrence graph.

For ESim, Metapath2vec and HIN2vec, we construct an HIN by viewing Figure 2 as the HIN schema. Based on the schema, we select five meta-paths *doc-user-doc*, *doc-label-doc*, *doc-word-doc*, *doc-tag-doc* and *word-word* to guide the embedding. (On Amazon, we replace *doc-tag-doc* with *doc-product-doc*.) After the node embedding step, we get representation vectors of each word/tag. Then, we train a CNN classifier using original training data $\{\mathcal{D}_l : l \in \mathcal{L}\}$ and the pre-trained HIN embeddings.

For all baselines using the CNN classifier (i.e., CNN, WeSTClass, ESim, Metapath2vec and HIN2vec), we adopt the architecture in Section 3.4 (the same as MetaCat) to align the experiment settings. For HAN, we use a forward GRU with 100 dimension for both word and sentence encoding. The training process of all neural classifiers is performed using SGD with a batch size of 256. The dimension of all embedding vectors is 100 (except BERT whose base model is set to be 768-dimensional).

For MetaCat, we set the local context window size $h = 5$, the document-specific vocabulary size $|\mathcal{N}(\boldsymbol{e}_d)| = 50$, and the number of generated training samples per class $|\mathcal{D}_l^*| = 100$.

4.3 Performance Comparison

Tables 2 and 3 demonstrate the Micro and Macro F1 scores of compared methods on the five datasets. We repeat each experiment 5 times with the mean and standard deviation reported. We cannot get the performance of TextGCN on GitHub-Sec and Amazon because the model (i.e., the constructed graph) is too large to fit into our GPU with 11GB memory. (The Twitter dataset can fit because it has a much smaller average document length.)

As we can observe from Tables 2 and 3: (1) MetaCat consistently outperforms all baselines by a clear margin on all datasets. It achieves a 4.3% *absolute* improvement on average in comparison with TextGCN, the second best approach in our table. When comparing with other baselines, our absolute improvement is over 10%. (2) In contrast to methods using plain text embeddings (i.e., CNN, HAN and WeSTClass), the performance boosts of MetaCat are more significant on smaller datasets (i.e., GitHub-Bio and GitHub-AI). In fact, when the corpus is small, word2vec cannot generate high-quality word embeddings. Consequently, neural models using

[2]Our code and datasets are available at `https://github.com/yuzhimanhua/MetaCat`.

Table 2: Micro F1 scores of compared algorithms on the five datasets. "–": excessive memory requirements.

Type	Method	GitHub-Bio	GitHub-AI	GitHub-Sec	Amazon	Twitter
Text-based	CNN [12]	0.2227 ± 0.0195	0.2404 ± 0.0404	0.4909 ± 0.0489	0.4915 ± 0.0374	0.3106 ± 0.0613
	HAN [38]	0.1409 ± 0.0145	0.1900 ± 0.0299	0.4677 ± 0.0334	0.4809 ± 0.0372	0.3163 ± 0.0878
	PTE [32]	0.3170 ± 0.0516	0.3511 ± 0.0403	0.4551 ± 0.0249	0.2997 ± 0.0786	0.1945 ± 0.0250
	WeSTClass [23]	0.3680 ± 0.0138	0.5036 ± 0.0287	0.6146 ± 0.0084	0.5312 ± 0.0161	0.3568 ± 0.0178
	PCEM [36]	0.3426 ± 0.0160	0.4820 ± 0.0292	0.5912 ± 0.0341	0.4645 ± 0.0163	0.2387 ± 0.0344
	BERT [4]	0.2680 ± 0.0303	0.2451 ± 0.0273	0.5538 ± 0.0368	0.5240 ± 0.0261	0.3312 ± 0.0860
Graph-based	ESim [27]	0.2925 ± 0.0223	0.4376 ± 0.0323	0.5480 ± 0.0109	0.5320 ± 0.0246	0.3512 ± 0.0226
	Metapath2vec [5]	0.3956 ± 0.0141	0.4444 ± 0.0231	0.5772 ± 0.0594	0.5256 ± 0.0335	0.3516 ± 0.0407
	HIN2vec [6]	0.2564 ± 0.0131	0.3614 ± 0.0234	0.5218 ± 0.0466	0.4987 ± 0.0252	0.2944 ± 0.0614
	TextGCN [39]	0.4759 ± 0.0126	0.6353 ± 0.0059	–	–	0.3361 ± 0.0032
	MetaCat	**0.5258 ± 0.0090**	**0.6889 ± 0.0128**	**0.7243 ± 0.0336**	**0.6422 ± 0.0058**	**0.3971 ± 0.0169**

Table 3: Macro F1 scores of compared algorithms on the five datasets. "–": excessive memory requirements.

Type	Method	GitHub-Bio	GitHub-AI	GitHub-Sec	Amazon	Twitter
Text-based	CNN [12]	0.1896 ± 0.0133	0.1796 ± 0.0216	0.4268 ± 0.0584	0.5056 ± 0.0376	0.2858 ± 0.0559
	HAN [38]	0.0677 ± 0.0208	0.0961 ± 0.0254	0.4095 ± 0.0590	0.4644 ± 0.0597	0.2592 ± 0.0826
	PTE [32]	0.2630 ± 0.0371	0.3363 ± 0.0250	0.3803 ± 0.0218	0.2563 ± 0.0810	0.1739 ± 0.0190
	WeSTClass [23]	0.3414 ± 0.0129	0.4056 ± 0.0248	0.5497 ± 0.0054	0.5234 ± 0.0147	0.3085 ± 0.0398
	PCEM [36]	0.2977 ± 0.0281	0.3751 ± 0.0350	0.4033 ± 0.0336	0.4239 ± 0.0237	0.2039 ± 0.0472
	BERT [4]	0.1740 ± 0.0164	0.2083 ± 0.0415	0.4956 ± 0.0164	0.4911 ± 0.0544	0.2834 ± 0.0550
Graph-based	ESim [27]	0.2598 ± 0.0182	0.3209 ± 0.0202	0.4672 ± 0.0171	0.5336 ± 0.0220	0.3399 ± 0.0113
	Metapath2vec [5]	0.3214 ± 0.0128	0.3220 ± 0.0290	0.5140 ± 0.0637	0.5239 ± 0.0437	0.3443 ± 0.0208
	HIN2vec [6]	0.2742 ± 0.0136	0.2513 ± 0.0211	0.4000 ± 0.0115	0.4261 ± 0.0284	0.2411 ± 0.0142
	TextGCN [39]	0.4817 ± 0.0078	0.5997 ± 0.0013	–	–	0.3191 ± 0.0029
	MetaCat	**0.5230 ± 0.0080**	**0.6154 ± 0.0079**	**0.6323 ± 0.0235**	**0.6496 ± 0.0091**	**0.3612 ± 0.0067**

word2vec embeddings will not achieve satisfying performance. In this case, leveraging multi-modal signals in representation learning becomes necessary. (3) Despite its great success in supervised tasks, BERT is not suitable for our task without sufficient training data, probably because the language style of GitHub files and tweets are different from that of Wikipedia, which might require strong supervision for fine-tuning. (4) Although HIN embedding techniques (i.e., ESim, Metapath2vec and HIN2vec) consider data heterogeneity and utilize unlabeled data during the embedding step, they still do not address the label scarcity bottleneck when training the classifier. This leads to their inferiority towards MetaCat.

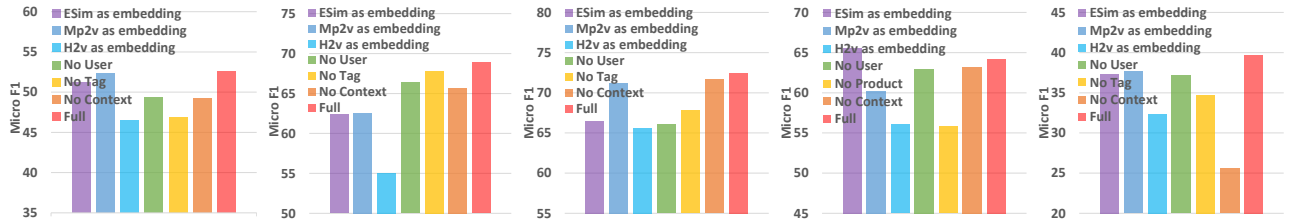
4.4 Effect of Embedding Learning

Embedding Method. Tables 2 and 3 tell us that our generation-guided embedding module, *together with training data generation*, can outperform several HIN embedding baselines. Now, to further explore the effectiveness of the proposed embedding technique, we perform a "fairer" comparison by fixing all the other parts in MetaCat and vary the embedding module only.

To be specific, we can use ESim, Metapath2vec and HIN2vec to replace our current embedding technique, which generates three ablations ESim-as-embedding, Mp2v-as-embedding and H2v-as-embedding. Moreover, we exploit various metadata as well as word context information during the embedding process. To demonstrate their contributions, we create three ablations No-User, No-Context, and No-Tag (No-Product for the Amazon dataset). Here No-User means we do not consider user information during embedding. Similar meanings can be inferred for the other two ablations. Note that our generation step needs $\boldsymbol{e}_l$ and $\boldsymbol{e}_w$, so there is no so-called "No-Label" or "No-Word". Figures 3 and 4 show the performance of these variants and our Full model.

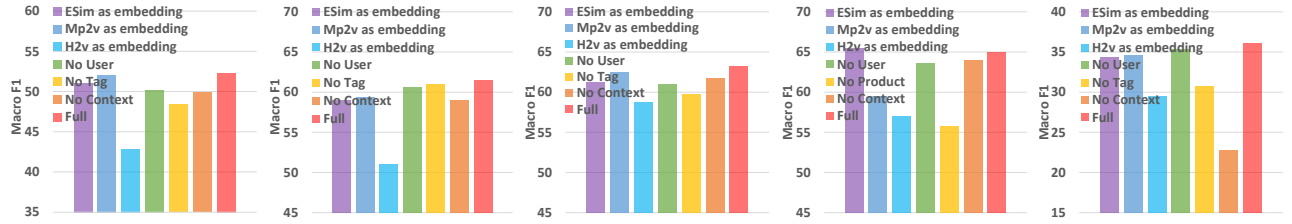
We have the following observations: (1) Full outperforms ESim-as-embedding, Mp2v-as-embedding and H2v-as-embedding on almost all datasets, with only one exception where ESim-as-embedding performs the best on Amazon. On the other four datasets, the average *absolute* improvement of Full in comparison with ESim-as-embedding (resp., Mp2v-as-embedding) is 4.0% (resp., 2.5%) in terms of Micro F1. This finding demonstrates the advantage of our generation-guided embedding over meta-path based HIN embedding in this task. As mentioned above, ESim and Metapath2vec rely on meta-path guided random walks to model higher-order relationships between nodes. However, in our task, there is an evident generative process, with which we believe that characterizing generative relationships between various elements is more important than describing long-distance connections. (2) Full consistently outperforms the ablations ignoring different metadata, indicating that users, tags, products and contexts all play a positive role in classification. Meanwhile, their importance varies in different datasets. For example, product information is extremely important on Amazon according to Figure 3(d). This is intuitive since Amazon's topic labels are essentially product categories. In contrast, user information is more useful on GitHub than it is on Amazon. This can be explained by the following statistics: GitHub-Bio has 351 pairs of documents sharing the same user, out of which 288 (82%) have the same label; GitHub-AI has 348 pairs of repositories having the same user, among which 217 (62%) belong to the same class.

Embedding Dimension. Next, we investigate the performance of MetaCat with respect to its embedding dimension. Figure 5



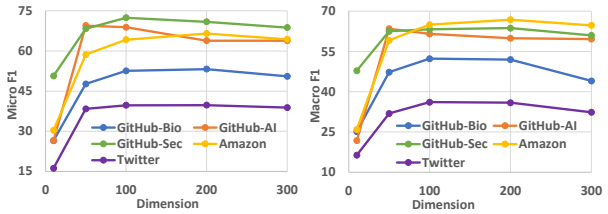
(a) GitHub-Bio (b) GitHub-AI (c) GitHub-Sec (d) Amazon (e) Twitter

Figure 3: Micro F1 scores of algorithms with different embedding modules.



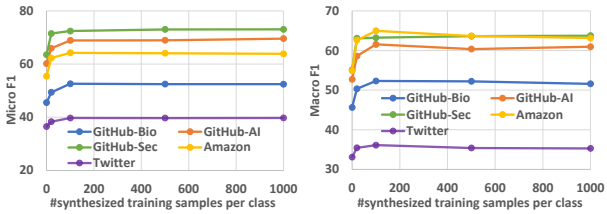
(a) GitHub-Bio (b) GitHub-AI (c) GitHub-Sec (d) Amazon (e) Twitter

Figure 4: Macro F1 scores of algorithms with different embedding modules.



(a) Micro F1 (b) Macro F1

Figure 5: Performance of MetaCat with respect to the embedding dimension.



(a) Micro F1 (b) Macro F1

Figure 6: Performance of MetaCat with respect to the number of synthesized training samples.

Table 4: Most similar words to a given label in the embedding space of MetaCat.

Dataset	Label l	Top similar words w
GitHub-Bio	Genetics	mega, ezmap, igess, phenotypesimulator, multigems
	Structural Bioinformatics	pywater, msisensor, knotty, breakpointsurveyor, cmv
GitHub-AI	Entity Recognition	ner, ld, conll, entity, tags
	Speech Synthesis	tacotron, lecture, tts, lj, spectrogram
Amazon	Home & Kitchen	spinner, lettuce, salad, spinning, greens
	Movies & TV	mafia, undercover, warren, depp, pacino
Twitter	Travel & Transport	baggage, airway, claim, fdny, jetblue
	Shop & Service	keyfood, greenmarket, unsqgreenmarket, nailsbymii, nailart

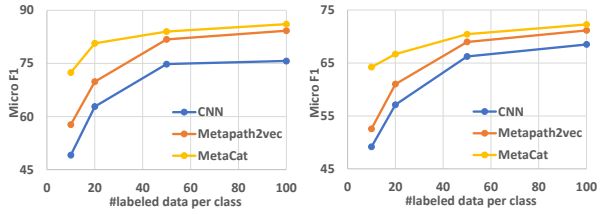
reports Micro and Macro F1 scores with 10, 50, 100, 200 and 300-dimensional embedding vectors. We can see that the performance drops when the dimension becomes too large. This observation is aligned with the results in [27, 33]. In fact, too small dimension cannot sufficiently capture the semantics, while too large dimension may lead to some overfitting problems, especially under weak supervision. Figure 5 shows that setting the dimension as 100 is reasonable in our experiments.

Cases. Table 4 shows the top-5 similar words to a given label in the embedding space of MetaCat. Here the similarity between word w and label l is defined as $\cos(\boldsymbol{e}_w, \boldsymbol{e}_l)$. Due to space limit, we demonstrate 8 categories from 4 datasets. We can find many strong topic indicators in Table 4. For example, in the GitHub domain, there are system/tool names (e.g., "*pywater*" and "*tacotron*") and dataset names (e.g., "*conll*" and "*lj*") related to the given category; in the Twitter domain, the top similar words mainly reflect the function of POIs (e.g., "*airway*", "*jetblue*" and "*unsqgreenmarket*").

4.5 Effect of Training Data Generation

We generate 100 training samples for each class in all previous experiments. To investigate the effect of synthesized training data amount, we plot the performance of MetaCat with 0, 20, 100, 500 and 1000 generated samples per class in Figure 6.

When $|\mathcal{D}_l^*| < 100$, the F1 scores increase evidently with $|\mathcal{D}_l^*|$. For example, the average *absolute* improvement of Micro F1 is 7.3% on the five datasets when comparing $|\mathcal{D}_l^*| = 100$ with $|\mathcal{D}_l^*| = 0$ (i.e., using "real" training data only). This observation validates our claim that incorporating generated training data can boost classification performance. However, when the number of documents becomes larger, the performance change is quite subtle. In fact, the



(a) GitHub-Sec (b) Amazon

	CNN	Metapath2vec	MetaCat
Embed Metadata?	***No***	***Yes***	***Yes***
Generate Training Samples?	***No***	***No***	***Yes***

(c) Comparisons of the three approaches

Figure 7: Micro F1 scores of CNN, Metapath2vec and MetaCat with respect to the number of "real" training samples.

fluctuation is less than 1% in most cases after $|\mathcal{D}_l^*| \geq 100$. Moreover, generating too many data will make the training process inefficient. In all, we believe having 100 to 500 synthesized training samples per class will strike a good balance in our task.

4.6 Effect of "Real" Training Data

According to Sections 4.4 and 4.5, we already know both embedding and generation modules are helpful when supervision is minimal. Now we examine their effects as "real" training data is increased. Three approaches are picked: CNN (does not embed metadata; does not generate training samples), Metapath2vec (embeds metadata; does not generate training samples) and MetaCat (embeds metadata; generates training samples). Figure 7 shows their performance with 10, 20, 50 and 100 "real" training samples per class on GitHub-Sec and Amazon.

Let us first compare MetaCat and Metapath2vec. When the amount of labeled data is relatively large, MetaCat only outperforms Metapath2vec by a small margin. However, if fewer labeled documents are provided, Metapath2vec exhibits more evident performance drop. For example, when $|\mathcal{D}_l| = 10$, the gap between MetaCat and Metapath2vec is 14.7% on GitHub-Sec; when $|\mathcal{D}_l| = 100$, the gap becomes 1.8%. This observation shows that training data generation is more powerful when supervision is weaker.

Then we proceed to the curves of CNN. Similar to Metapath2vec, its performance drops drastically when $|\mathcal{D}_l|$ becomes small. However, there is always a large gap between CNN and MetaCat. When labeled training samples are sufficient, this gap is mainly attributed to the usage of metadata in embedding (because we already show that training data generation plays a limited role in this case); when there is less supervision, the gap becomes a composite effect of utilizing metadata and generating synthesized samples. Different from training data generation, no matter how many labeled documents are involved, exploiting metadata in embedding is always helpful.

5 RELATED WORK

Weakly Supervised Text Classification. Two forms of supervisions are commonly studied under weakly supervised text classification: (1) *class-related keywords*. For example, dataless classification [1] only relies on descriptive keywords and has no requirement of any labeled document. Subsequent studies either extend topic models [2, 14, 16, 18] or exploit embedding techniques [15, 22] and contextualized representation learning models [21] to incorporate such keyword information. (2) *a small set of labeled documents*. For example, [32] jointly learns word/document/label representations; [23] generates pseudo documents to help the training process; [39] applies graph convolutional networks on the word-document co-occurrence graph. [24] and [36] further study weakly supervised hierarchical classification. However, these studies focus on text without metadata, which restricts their capacity in some practical scenarios. In contrast, MetaCat goes beyond plain text classification and utilizes multi-modal signals.

Text Classification with Metadata. Existing studies apply metadata to improve the performance of a text classifier, such as user and product information in sentiment analysis [31], author information in paper topic classification [26], and user biography data in tweet localization [41]. However, each of these frameworks focuses on one specific type of data, while we propose a general framework to deal with various data sources and metadata types. Kim et al. [11] study how to "inject" categorical metadata information into neural text classifiers. Their model can be applied in many cases such as review sentiment classification and paper acceptance classification. However, the model is designed under fully supervised settings and does not tackle label scarcity.

Embedding Learning of Heterogeneous Data. Heterogeneous Information Network (HIN) embedding is the most common technique to encode multi-modal signals. Many HIN embedding methods [5, 6, 27] leverage meta-path guided random walks to jointly model multiple interactions in a latent embedding space. One can refer to a recent tutorial [28] for more related studies. We have discussed the differences between HIN embedding and our generation-guided embedding in Section 3.2. From the view of applications, several studies apply HIN embeddings into downstream classification tasks such as malware detection [9] and medical diagnosis [8]. Both [9] and [8], as well as experiments in [5, 6, 27], deal with structured data under fully supervised settings.

6 CONCLUSIONS AND FUTURE WORK

We presented MetaCat, a framework to categorize text with metadata under minimal supervision. To tackle the challenges of data heterogeneity and label scarcity, we propose a generation-guided embedding module and a training data generation module. Both modules are derived from a generative process characterizing the relationships between text and metadata. We demonstrate the effectiveness of MetaCat on five datasets. Moreover, we validate the design of our framework by showing (1) the superiority of our embedding module towards meta-path based HIN embedding methods and (2) the significant contribution of our generation module especially when the amount of supervision is very limited.

For future work, first, it is interesting to study how to effectively integrate different forms of supervision (e.g., annotated documents and class-related keywords) to further boost the performance. Second, we would like to explore the possibility of coupling heterogeneous signal embedding and graph neural networks during the classification process.

ACKNOWLEDGMENTS

We thank Sha Li for useful discussions. The research was sponsored in part by DARPA under Agreements No. W911NF-17-C-0099 and FA8750-19-2-1004, National Science Foundation IIS 16-18481, IIS 17-04532, and IIS-17-41317, and DTRA HDTRA11810026. Any opinions, findings, and conclusions or recommendations expressed in this document are those of the author(s) and should not be interpreted as the views of any U.S. Government. The U.S. Government is authorized to reproduce and distribute reprints for Government purposes notwithstanding any copyright notation hereon. We thank anonymous reviewers for valuable and insightful feedback.

REFERENCES

[1] Ming-Wei Chang, Lev-Arie Ratinov, Dan Roth, and Vivek Srikumar. 2008. Importance of Semantic Representation: Dataless Classification.. In *AAAI'08*. 830–835.
[2] Xingyuan Chen, Yunqing Xia, Peng Jin, and John Carroll. 2015. Dataless text classification with descriptive LDA. In *AAAI'15*.
[3] Zhiyuan Cheng, James Caverlee, and Kyumin Lee. 2010. You are where you tweet: a content-based approach to geo-locating twitter users. In *CIKM'10*. 759–768.
[4] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In *NAACL-HLT'19*. 4171–4186.
[5] Yuxiao Dong, Nitesh V Chawla, and Ananthram Swami. 2017. metapath2vec: Scalable representation learning for heterogeneous networks. In *KDD'17*. 135–144.
[6] Tao-yang Fu, Wang-Chien Lee, and Zhen Lei. 2017. Hin2vec: Explore meta-paths in heterogeneous information networks for representation learning. In *CIKM'17*. 1797–1806.
[7] Siddharth Gopal and Yiming Yang. 2014. Von mises-fisher clustering models. In *ICML'14*. 154–162.
[8] Anahita Hosseini, Ting Chen, Wenjun Wu, Yizhou Sun, and Majid Sarrafzadeh. 2018. HeteroMed: Heterogeneous Information Network for Medical Diagnosis. In *CIKM'18*. 763–772.
[9] Shifu Hou, Yanfang Ye, Yangqiu Song, and Melih Abdulhayoglu. 2017. Hindroid: An intelligent android malware detection system based on structured heterogeneous information network. In *KDD'17*. 1507–1515.
[10] Rie Johnson and Tong Zhang. 2017. Deep pyramid convolutional neural networks for text categorization. In *ACL'17*. 562–570.
[11] Jihyeok Kim, Reinald Kim Amplayo, Kyungjae Lee, Sua Sung, Minji Seo, and Seung-won Hwang. 2019. Categorical Metadata Representation for Customized Text Classification. *TACL* 7 (2019), 201–215.
[12] Yoon Kim. 2014. Convolutional Neural Networks for Sentence Classification. In *EMNLP'14*. 1746–1751.
[13] Omer Levy, Yoav Goldberg, and Ido Dagan. 2015. Improving distributional similarity with lessons learned from word embeddings. *TACL* 3 (2015), 211–225.
[14] Chenliang Li, Jian Xing, Aixin Sun, and Zongyang Ma. 2016. Effective document labeling with very few seed words: A topic model approach. In *CIKM'16*. 85–94.
[15] Keqian Li, Hanwen Zha, Yu Su, and Xifeng Yan. 2018. Unsupervised neural categorization for scientific publications. In *SDM'18*. 37–45.
[16] Ximing Li, Changchun Li, Jinjin Chi, Jihong Ouyang, and Chenliang Li. 2018. Dataless text classification: A topic modeling approach with document manifold. In *CIKM'18*. 973–982.
[17] Jingzhou Liu, Wei-Cheng Chang, Yuexin Wu, and Yiming Yang. 2017. Deep learning for extreme multi-label text classification. In *SIGIR'17*. 115–124.
[18] Yue Lu and Chengxiang Zhai. 2008. Opinion integration through semi-supervised topic modeling. In *WWW'08*. 121–130.
[19] Kanti V Mardia and Peter E Jupp. 2009. *Directional statistics*. Vol. 494. John Wiley & Sons.
[20] Julian McAuley and Jure Leskovec. 2013. Hidden factors and hidden topics: understanding rating dimensions with review text. In *RecSys'13*. 165–172.
[21] Dheeraj Mekala and Jingbo Shang. 2020. Contextualized Weak Supervision for Text Classification.. In *ACL'20*.
[22] Yu Meng, Jiaxin Huang, Guangyuan Wang, Zihan Wang, Chao Zhang, Yu Zhang, and Jiawei Han. 2020. Discriminative Topic Mining via Category-Name Guided Text Embedding. In *WWW'20*. 2121–2132.
[23] Yu Meng, Jiaming Shen, Chao Zhang, and Jiawei Han. 2018. Weakly-supervised neural text classification. In *CIKM'18*. 983–992.
[24] Yu Meng, Jiaming Shen, Chao Zhang, and Jiawei Han. 2019. Weakly-supervised hierarchical text classification. In *AAAI'19*. 6826–6833.
[25] Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg S Corrado, and Jeff Dean. 2013. Distributed representations of words and phrases and their compositionality. In *NIPS'13*. 3111–3119.
[26] Michal Rosen-Zvi, Thomas Griffiths, Mark Steyvers, and Padhraic Smyth. 2004. The author-topic model for authors and documents. In *UAI'04*. 487–494.
[27] Jingbo Shang, Meng Qu, Jialu Liu, Lance M Kaplan, Jiawei Han, and Jian Peng. 2016. Meta-path guided embedding for similarity search in large-scale heterogeneous information networks. *arXiv preprint arXiv:1610.09769* (2016).
[28] Chuan Shi and Philip S Yu. 2019. Recent Developments of Deep Heterogeneous Information Network Analysis. In *CIKM'19*. 2973–2974.
[29] Chi Sun, Xipeng Qiu, Yige Xu, and Xuanjing Huang. 2019. How to fine-tune BERT for text classification?. In *CCL'19*. 194–206.
[30] Yizhou Sun, Jiawei Han, Xifeng Yan, Philip S Yu, and Tianyi Wu. 2011. Pathsim: Meta path-based top-k similarity search in heterogeneous information networks. *PVLDB* 4, 11 (2011), 992–1003.
[31] Duyu Tang, Bing Qin, and Ting Liu. 2015. Learning semantic representations of users and products for document level sentiment classification. In *ACL'15*. 1014–1023.
[32] Jian Tang, Meng Qu, and Qiaozhu Mei. 2015. Pte: Predictive text embedding through large-scale heterogeneous text networks. In *KDD'15*. 1165–1174.
[33] Jian Tang, Meng Qu, Mingzhe Wang, Ming Zhang, Jun Yan, and Qiaozhu Mei. 2015. Line: Large-scale information network embedding. In *WWW'15*. 1067–1077.
[34] Chenglong Wang, Feijun Jiang, and Hongxia Yang. 2017. A hybrid framework for text modeling with convolutional rnn. In *KDD'17*. 2061–2069.
[35] Ke Wang and Xiaojun Wan. 2018. Sentiment analysis of peer review texts for scholarly papers. In *SIGIR'18*. 175–184.
[36] Huiru Xiao, Xin Liu, and Yangqiu Song. 2019. Efficient Path Prediction for Semi-Supervised and Weakly Supervised Hierarchical Text Classification. In *WWW'19*. 3370–3376.
[37] Chao Xing, Dong Wang, Chao Liu, and Yiye Lin. 2015. Normalized word embedding and orthogonal transform for bilingual word translation. In *NAACL-HLT'15*. 1006–1011.
[38] Zichao Yang, Diyi Yang, Chris Dyer, Xiaodong He, Alex Smola, and Eduard Hovy. 2016. Hierarchical attention networks for document classification. In *NAACL'16*. 1480–1489.
[39] Liang Yao, Chengsheng Mao, and Yuan Luo. 2019. Graph convolutional networks for text classification. In *AAAI'19*. 7370–7377.
[40] Chao Zhang, Keyang Zhang, Quan Yuan, Fangbo Tao, Luming Zhang, Tim Hanratty, and Jiawei Han. 2017. React: Online multimodal embedding for recency-aware spatiotemporal activity modeling. In *SIGIR'17*. 245–254.
[41] Yu Zhang, Wei Wei, Binxuan Huang, Kathleen M Carley, and Yan Zhang. 2017. RATE: Overcoming Noise and Sparsity of Textual Features in Real-Time Location Estimation. In *CIKM'17*. 2423–2426.
[42] Yu Zhang, Frank F. Xu, Sha Li, Yu Meng, Xuan Wang, Qi Li, and Jiawei Han. 2019. HiGitClass: Keyword-Driven Hierarchical Classification of GitHub Repositories. In *ICDM'19*. 876–885.
[43] Chunting Zhou, Chonglin Sun, Zhiyuan Liu, and Francis Lau. 2015. A C-LSTM neural network for text classification. *arXiv preprint arXiv:1511.08630* (2015).